

Troubleshoot apps on Azure Kubernetes Service

In this exercise, you deploy a containerized API to Azure Kubernetes Service (AKS) and then diagnose and resolve common Kubernetes issues. You use **kubectl** commands to identify problems, inspect pod status, check logs, and view events. You then use **kubectl edit** to fix misconfigurations including Service selector mismatches, missing environment variables, and invalid readiness probe paths.

Tasks performed in this exercise:

- Download the project starter files
- Deploy resources to Azure (ACR, AKS cluster)
- Diagnose and resolve some common issues
- Clean up Azure resources

This exercise takes approximately **30** minutes to complete.

 **Important:** Azure Container Registry task runs are temporarily paused from Azure free credits. This exercise requires a Pay-As-You-Go, or another paid plan.

Before you start

To complete the exercise, you need:

- An Azure subscription with the permissions to deploy the necessary Azure services. If you don't already have one, you can [sign up for one](#).
- [Visual Studio Code](#) on one of the [supported platforms](#).
- The latest version of the [Azure CLI](#).
- The Kubernetes command-line tool, [kubectl](#).
- [Python 3.12](#) or greater.

Download project starter files and deploy Azure services

In this section you download the starter files for the console app and use a script to deploy the necessary services to your Azure subscription. The Azure resource deployments can take 10-15 minutes to complete.

1. Open a browser and enter the following URL to download the starter file. The file will be saved in your default download location.

CodeCopy

```
https://github.com/MicrosoftLearning/mslearn-azure-ai/raw/main/downloads/python/aks-troubleshoot-python.zip
```

2. Copy, or move, the file to a location in your system where you want to work on the project. Then unzip the file into a folder.
3. Launch Visual Studio Code (VS Code) and select **File > Open Folder...** in the menu, then choose the folder containing the project files.
4. Open the *azdeploy.py* deployment script and change the resource group and location values at the top of the script to meet your needs, then save your changes. **Note:** Do not change anything else in the script.

CodeCopy

```
rg = "<your-resource-group-name>" # Resource Group name
location = "<your-azure-region>" # Azure region for the resources
```

5. In the menu bar select **Terminal > New Terminal** to open a terminal window in VS Code.
6. Run the following command to login to your Azure account. Answer the prompts to select your Azure account and subscription for the exercise.

CodeCopy

```
az login
```

7. Run the following commands to ensure your subscription has the necessary resource providers to install AKS and ACR. The **Microsoft.Compute**, **Microsoft.Network**, and **Microsoft.Storage** providers are dependencies of AKS that Azure usually registers automatically, but explicitly registering them avoids occasional cluster creation failures on new subscriptions.

CodeCopy

```
az provider register --namespace Microsoft.ContainerService
az provider register --namespace Microsoft.ContainerRegistry
az provider register --namespace Microsoft.Compute
az provider register --namespace Microsoft.Network
az provider register --namespace Microsoft.Storage
```

8. Make sure you are in the root directory of the project and run the following command to launch the deployment script.

CodeCopy

```
python azdeploy.py
```

⚠ **Note:** Depending on your system configuration, the Python command might be **python3** instead of **python**.

Deploy resources to Azure

With the deployment script running, follow these steps to create the needed resources in Azure.

1. Enter **1** to launch **Create Azure Container Registry (ACR)**. This creates the resource where the API container will be stored, and later pulled into the AKS resource.
2. After the ACR resource has been created, enter **2** to launch **Build and push API image to ACR**. This option uses ACR tasks to build the image and add it to the ACR repository. This operation can take 3-5 minutes to complete.
3. After the image has been built and pushed to ACR, enter **3** to launch the **Create AKS cluster** option. This creates the AKS resource configured with a managed identity, and gives the service permission to pull images from the ACR resource.
4. After the AKS cluster deployment has completed, enter **4** to launch the **Get AKS credentials for kubectl** option. This uses the **az aks get-credentials** command to retrieve credentials and configure **kubectl**.
5. After the credentials have been set, enter **5** to launch the **Deploy application to AKS** option. This deploys an API to the AKS cluster.
6. After the app has been deployed, enter **6** to launch the **Check deployment status** option. This option reports if each of the resources have been successfully deployed.

If all of the services return a **successful** message, enter **8** to exit the deployment script.

If the AKS cluster is in a **Failed** or **Canceled** state, correct the reported issue, then enter **7** to launch the **Delete failed AKS deployment** option before running option **3** again. This guarded option does not delete a healthy or in-progress cluster.

Note: Leave the terminal open, all of the steps in the exercise are performed in the terminal.

Troubleshoot the deployment

The deployment script created all Kubernetes resources in a **namespace** called **aks-troubleshoot**. Namespaces are a way to organize and isolate resources within a Kubernetes cluster. They allow you to group related resources together, apply resource quotas, and manage access control. When you don't specify a namespace, resources are created in the **default** namespace. For this exercise, all **kubectl** commands include **-n aks-troubleshoot** to target the correct namespace.

After you verify the deployment, you work through three troubleshooting scenarios. In each scenario, you apply a manifest file that introduces a specific error into the deployment. Then you use **kubectl** commands to diagnose the problem and edit the configuration to resolve it.

Verify the deployment

In this section you confirm the application deployed by the setup script is running correctly before introducing errors.

1. Run the following command to verify the pod is running in the namespace. The command should return one pod with **Running** status and **1/1** in the READY column.

Code	Copy
<pre>kubectl get pods -n aks-troubleshoot</pre>	

2. Run the following command to verify the Service has endpoints. The command should return one endpoint slice listed with an IP address.

Code	Copy
<pre>kubectl get endpointslices -l kubernetes.io/service-name=api-service -n aks-troubleshoot</pre>	

3. Run the following command to test connectivity using port-forward. This command creates a tunnel from your local machine to the Service running in the cluster, allowing you to access it at <http://localhost:8080>.

Code	Copy
<pre>kubectl port-forward service/api-service 8080:80 -n aks-troubleshoot</pre>	

4. In the menu bar select **Terminal > New Terminal** to open a second terminal window in VS Code. Run the following command to test the connection. You should receive a JSON response with **"status": "healthy"**.

Code	Copy
<pre># Bash curl http://localhost:8080/healthz</pre>	

Code	Copy
<pre># PowerShell Invoke-RestMethod http://localhost:8080/healthz</pre>	

5. Switch back to the terminal where **port-forward** is running and enter **ctrl+c** to exit the command.

You verified the deployment is working correctly, next you diagnose a label mismatch issue.

Diagnose a label mismatch

A Service routes traffic to pods based on label selectors. When labels don't match, the Service has no endpoints and requests fail. The API was deployed with pods labeled **app: api** and a Service selector matching **app: api**. In this section you apply a Service configuration that changes the selector to **app: api-v2**, breaking the connection.

1. Run the following command to apply the Service configuration that creates a label mismatch error.

Code	Copy
<pre>kubectl apply -f k8s/label-mismatch-service.yaml -n aks-troubleshoot</pre>	

2. Run the following command to verify the pod is still running. The pod shows **Running** status with **1/1** ready and labels showing **app=api**.

Code	Copy
<pre>kubectl get pods --show-labels -n aks-troubleshoot</pre>	

3. Run the following command to check the Service endpoint slices. The command should return an endpoint slice with **<unset>** in the ENDPOINTS column, indicating no pods match the Service selector.

Code	Copy
<pre>kubectl get endpointslices -l kubernetes.io/service-name=api-service -n aks-troubleshoot</pre>	

4. Run the following command to view the Service details. Look for the **Selector** field in the output, which now shows **app=api-v2**.

Code	Copy
<pre>kubectl describe service api-service -n aks-troubleshoot</pre>	

This confirms the label mismatch. The Service selector is **app=api-v2** but the pod label is **app=api**.

5. Run the following command to open the Service configuration in an editor.

Code	Copy
<pre>kubectl edit service api-service -n aks-troubleshoot</pre>	

Note: The **kubectl edit** command fetches the live resource configuration from the cluster and opens it in a local text editor. When you save and close the editor, kubectl automatically sends the changes back to the Kubernetes API server, which validates and applies them to the running cluster. The editor depends on your environment:

- **Bash:** Opens **vi** by default. Press **i** to enter insert mode, make your changes, press **Esc**, then type **:wq** and press **Enter** to save and exit. Type **:q!** to exit without saving.
- **PowerShell (Windows):** Opens **Notepad** by default. Make your changes, select **File > Save** (or **Ctrl+S**), then close the window. Closing without saving cancels the edit.

6. In the editor, find the **selector** section and change **app: api-v2** to **app: api**. Save the changes and close the editor.
7. Run the following command to verify the endpoint slice addresses are restored. The command should return an endpoint slice with an IP address listed.

Code	Copy
<pre>kubectl get endpointslices -l kubernetes.io/service-name=api-service -n aks-troubleshoot</pre>	

You fixed the label mismatch issue, next you diagnose a CrashLoopBackOff.

Diagnose a CrashLoopBackOff

When a container fails to start, Kubernetes repeatedly restarts it, resulting in **CrashLoopBackOff** status. Reading logs reveals why the application crashed.

1. Run the following command to apply the deployment configuration that removes the required **API_KEY** environment variable.

Code	Copy
<pre>kubectl apply -f k8s/crashloop-deployment.yaml -n aks-troubleshoot</pre>	

2. Run the following command to watch the pod status. After a few moments, the pod enters **CrashLoopBackOff**. Enter **ctrl-c** to exit the command.

Code	Copy
<pre>kubectl get pods -n aks-troubleshoot -w</pre>	

3. Run the following command to check the pod logs for the error message. You should see an error indicating the missing environment variable.

Code	Copy
<pre>kubectl logs -l app=api -n aks-troubleshoot</pre>	

4. Fix the issue by editing the Deployment to add the **API_KEY** environment variable.

Code	Copy
<pre>kubectl edit deployment api-deployment -n aks-troubleshoot</pre>	

In the editor, find the **containers** section under **spec.template.spec**. Locate the **name: api** line and add the **env** block directly below it, matching the indentation of **name**:

Code	Copy
------	------

```
name: api
env:
- name: API_KEY
  value: "demo-api-key-12345"
ports:
```

Save the changes and close the editor.

5. Run the following command to watch the pod status. After a few moments, the pod enters **Running**. Enter **ctrl-c** to exit the command.

Code	Copy
<pre>kubectl get pods -n aks-troubleshoot -w</pre>	

You solved the CrashLoopBackOff issue, next you diagnose a readiness probe failure.

Diagnose a readiness probe failure

When a readiness probe fails, the pod shows **Running** but **0/1** containers are ready. Kubernetes won't add the pod to Service endpoints until it passes the readiness check. With a rolling update strategy, the old working pod continues serving traffic while the new pod remains stuck in a not-ready state.

1. Run the following command to apply the deployment configuration that introduces a readiness probe failure. This applies an invalid path for the readiness check.

Code	Copy
<pre>kubectl apply -f k8s/probe-failure-deployment.yaml -n aks-troubleshoot</pre>	

2. Run the following command to check the pod status. You should see two pods: the new pod shows **Running** but **0/1** in the READY column, while the old pod remains **1/1** ready. The rolling update is blocked because the new pod never becomes ready.

Code	Copy
<pre>kubectl get pods -n aks-troubleshoot</pre>	

3. Run the following command to check for probe failure events. The command returns all **Unhealthy** events, which include both readiness and liveness probe failures. Look for a message indicating the readiness probe failed with a 404 status code.

Code	Copy
<pre>kubectl get events -n aks-troubleshoot --field-selector reason=Unhealthy</pre>	

4. Run the following command to fix the readiness probe by editing the Deployment to correct the path.

Code	Copy
<pre>kubectl edit deployment api-deployment -n aks-troubleshoot</pre>	

In the editor, find the **readinessProbe** section and change **path: /invalid-path** to **path: /healthz**. Save the changes and close the editor.

[Before you start](#)

[Download project starter files and deploy Azure services](#)

Troubleshoot the deployment

[Clean up resources](#)

[Troubleshooting](#)

5. Run the following command to verify the new pod becomes ready and the old pod terminates. You should see only one pod with **Running** status and **1/1** in the READY column.

Code	Copy
<pre>kubectl get pods -n aks-troubleshoot</pre>	

You diagnosed and solved a readiness probe error, next you verify end-to-end connectivity.

Verify end-to-end connectivity

After completing all troubleshooting scenarios, you confirm the application is fully functional.

1. Run the following command to use port-forward to access the Service.

Code	Copy
<pre>kubectl port-forward service/api-service 8080:80 -n aks-troubleshoot</pre>	

2. In the menu bar select **Terminal > New Terminal** to open a second terminal window in VS Code. Run the following commands to test all endpoints.

Code	Copy
<pre># Bash curl http://localhost:8080/healthz curl http://localhost:8080/readyz curl http://localhost:8080/api/info</pre>	

Code	Copy
<pre># PowerShell Invoke-RestMethod http://localhost:8080/healthz Invoke-RestMethod http://localhost:8080/readyz Invoke-RestMethod http://localhost:8080/api/info</pre>	

3. Run the following command to check the pod logs to see the requests.

Code	Copy
<pre>kubectl logs -l app=api -n aks-troubleshoot</pre>	

You verified the application is fully functional, next you clean up resources.

Clean up resources

Now that you finished the exercise, you should delete the cloud resources you created to avoid unnecessary resource usage.

1. Run the following command in the VS Code terminal to delete the resource group, and all resources in the group. Replace **<rg-name>** with the name you choose earlier in the exercise. The command will launch a background task in Azure to delete the resource group.

Code	Copy
<pre>az group delete --name <rg-name> --no-wait --yes</pre>	

⚠ **CAUTION:** Deleting a resource group deletes all resources contained within it. If you chose an existing resource group for this exercise, any existing resources outside the scope of this exercise will also be deleted.

Troubleshooting

If you encounter issues while setting up this exercise, try the following troubleshooting steps:

Check deployment status with the deployment script

- Run the deployment script and select option **6. Check deployment status** to verify the state of all deployed resources.
- This command checks ACR provisioning state, AKS cluster provisioning state, and Kubernetes resource availability.
- Use this output to identify which component may be causing issues.

Resolve AKS cluster creation failures

- Quota validation can fail before an AKS resource is created, while failures later in provisioning can leave the cluster in a **Failed** or **Canceled** state.
- If the error reports that **Standard_D2s_v7** is unavailable or that the region has insufficient capacity, exit with option **8**. Change **AKS_VM_SIZE** to one of the v5 or v6 fallback sizes listed near the top of the deployment script, or change the **location** value, and then run option **3. Create AKS cluster** again.
- If the error reports insufficient quota, select a region where your subscription has available Dsv5-family quota or request a quota increase. Changing regions only helps when the other region has sufficient quota.
- The AKS cluster uses the **location** configured in the script even when the resource group already exists in another region.
- If option **6** reports **Failed** or **Canceled**, correct the underlying issue and run option **7. Delete failed AKS deployment** before retrying option **3**. If no AKS resource was created, no deletion is needed.

ACR image pull errors

- If pods show **ImagePullBackOff** or **ErrImagePull** status, verify the ACR resource was created and the image was pushed successfully.
- Run the deployment script option **2. Build and push API image to ACR** again if needed.
- Confirm the AKS cluster has permission to pull from ACR by checking the deployment script output for successful role assignment.

kubectl connection issues

- If kubectl commands fail with connection errors, run the deployment script option **4. Get AKS credentials for kubectl** to refresh credentials.
- Verify the AKS cluster is running by checking the Azure portal or running **az aks show --resource-group <rg-name> --name <aks-name> --query provisioningState**.

Resetting the exercise

- If you need to start the troubleshooting scenarios over, run the deployment script option **5. Deploy application to AKS** to redeploy the original working configuration.
- This reapplies the base deployment and service files, resetting any changes made during the exercise.

